

Tynovate Studio

**Real-Time Network Packet Analyzer and Threat
Detection Engine**

Track A

**Muhammad Suleman
Programming Foundations**

Architecture Design Document: Real-Time Network Threat Detection Engine

1. System Overview

The Network Threat Detection Engine is a high-performance, multi-threaded Network Intrusion Detection System (NIDS) built in C++17. Its primary goal is to monitor live network traffic, inspect data packets at a low level, track active communication sessions, and automatically identify cyber threats (such as Port Scans and SYN Floods) in real time with sub-millisecond latency.

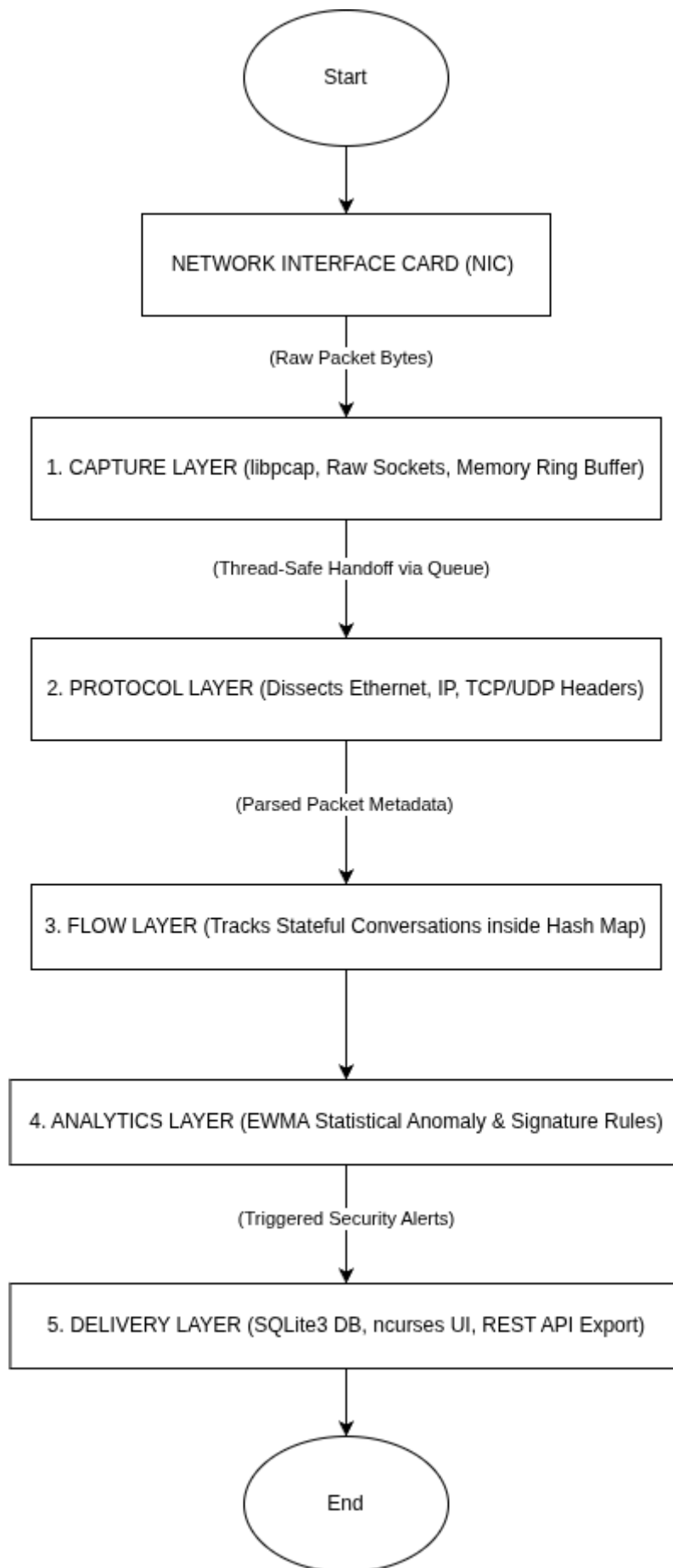
The system acts as an automated security guard for network interfaces, ensuring malicious behavior is caught and logged instantly without slowing down or dropping network packets.

2. Core Tools and Libraries Used

- **C++17:** For modern, high-performance, and type-safe systems programming.
- **libpcap:** To hook into Linux raw sockets and capture live network packets directly from the Network Interface Card (NIC).
- **CMake:** As the cross-platform build automation system.
- **POSIX Threads (pthread / std::thread):** To manage the multi-threaded pipeline and handle workload concurrency.
- **SQLite3:** For lightweight, persistent local logging of detected security alerts.
- **ncurses:** To build a real-time, interactive terminal dashboard for network administrators.

3. High-Level Component Layers

The engine is organized into **five modular layers**. Each layer has a single, dedicated responsibility.



4. Short Written Breakdown of Each Layer

Layer 1: Capture Layer

- **Purpose:** Captures raw byte streams passing through the selected network interface.
- **Mechanism:** It configures `libpcap` into **Promiscuous Mode** to read all network traffic. To avoid packet drops under heavy load, it allocates a 20MB kernel-level ring buffer and isolates the capture loop into a dedicated **Producer Thread**. This thread does no computational work except capturing packets and immediately placing them into a shared memory queue.

Layer 2: Protocol Layer (Dissection)

- **Purpose:** Translates raw packet binary data into human-readable network structures.
- **Mechanism:** It casts raw byte pointers to standard Linux networking structures. It extracts critical metadata by peeling away protocol layers step-by-step: Ethernet headers -> IP headers (Extracts Source/Destination IPs) TCP/UDP headers (Extracts Source/Destination Ports and Flags).

Layer 3: Flow Layer (Tracking Table)

- **Purpose:** Groups individual packets into stateful network sessions ("conversations").
- **Mechanism:** Instead of looking at packets in isolation, it aggregates them using a unique 5-tuple key: `[Source IP, Destination IP, Source Port, Destination Port, Protocol]`. These sessions are stored and updated inside a high-speed `std::unordered_map` to track things like active connection states and packet counters.

Layer 4: Analytics Layer (Threat Detection)

- **Purpose:** Analyzes network behavior to flag malicious activities.
- **Mechanism:** This layer runs two primary types of threat detectors:
 1. **Signature/Rule Detection:** Catches explicit attacks like SYN Floods by tracking unfinished TCP handshakes.
 2. **Statistical Anomaly Detection:** Uses an **Exponentially Weighted Moving Average (EWMA)** formula to calculate a dynamic baseline of normal traffic speed. If a host suddenly bursts past this baseline, it triggers a **Port Scan** or **Data Exfiltration** alert without relying on static, hardcoded thresholds.

Layer 5: Delivery Layer (Visualization & Alerts)

- **Purpose:** Logs threat alerts and displays current network metrics to system admins.

- **Mechanism:** When a threat is confirmed by the Analytics layer, this layer safely writes the alert signature, attacker IP, and timestamp into an **SQLite3 database**. Concurrently, it updates a terminal-based **ncurses dashboard** to show live throughput statistics and exposes a **REST API endpoint** for external SIEM log integrations.

5. System Data Flow

1. A packet arrives at the **Network Interface Card (NIC)**.
2. The **Producer Thread (Capture Layer)** intercepts the raw bytes via **libpcap** and pushes the packet into the safe shared queue buffer.
3. The **Consumer Thread (Protocol Layer)** pops the packet, extracts the IP and Port configurations, and creates a session record.
4. The **Flow & Analytics Layer** evaluates the packet against the historical EWMA baseline of that specific IP address.
5. If anomalous behavior is detected, an alert object is instantly created and dispatched to the **SQLite3 DB** and the **ncurses UI screen**.